AI Capstone Project 1 111550056 陳晉祿

Topic: Analysis and Prediction of Traffic Accident Categories

Motivation:

Traffic accidents occur daily for a variety of reasons and are often unavoidable. Major incidents frequently make headlines, raising the question: how can we minimize the resulting harm? Traffic accidents involving casualties are classified into two categories: A1, where fatalities occur, and A2, where injuries are sustained without any deaths. By leveraging predictive models to rapidly forecast the potential category of an accident, emergency responders and healthcare professionals can more effectively allocate resources, ultimately reducing casualties and mitigating harm.

Dataset Documentation: Taiwan Traffic Accident Data (A1 & A2)

Link: https://github.com/Luluboy168/Taiwan_traffic_accident_prediction

1. Overview

This dataset contains traffic accident records from Taiwan with information obtained from the open data website of Taiwan government. After process, it has been split into two main categories:

- **A1**: Fatal accidents (致死車禍)
- **A2**: Accidents involving only injuries (不含死亡之傷害事故)

Each row in the dataset represents a single accident event, with columns indicating various attributes associated with the accident such as date/time, weather, road condition and people involved in that accident.

---

2. External Source

- **Source**: 政府資料開放平台-歷史交通事故資料
- **Original Format**: CSV files containing detailed accident reports (in Chinese), and each row represented a single person involved in an accident with columns such as date, time, categories… Therefore, if multiple people were involved in one accident, the original data would have multiple rows with identical accident date/time/location fields but differing personal or vehicle information.

發生年度	發生月份	發生日期	發生時間	事故類別名稱	處理單位名稱警	發生地點	天候名稱	光線名稱	…
2024	1	20240101	500	A2	臺中市政府警察	臺中市大	晴	有照明且	…

---

3. Data Processing and Tools Used (for the detailed code, please check appendix)

1. **Original Data Download**
   - The original CSV files were downloaded from the 政府資料開放平台.
2. **Data analysis**

- Survey the information in the original data. And make sure format and the representation of the data.

3. **Data combining and cleaning**
    - Because the original data is divided by year, and also A2 category data is divided into 12 months a year. We need to read csv files then combine them from all different sources in order to do the cleaning and other processing to them
    - Many record in the original data have missing values, therefore a cleaning is necessary to ensure the dataset integrity.
4. **Mapping to Numeric Codes**
    - For easier usage in machine learning, I map the values of categorical labels to integer.
5. **Making dataset**
    - The original data contained detailed accident reports where each row represented a single person involved in an accident. But as we can imagine, this kind of information is not really good or enough to predict if a accident is fatal. For example, we have no way of knowing what the consequences would be if a 20-year-old boy got into an accident while riding a motorcycle in a sunny day. A better way is by checking how many people and what kind of vehicles have involved in the accident. So, I aggregate the relevant attributes (e.g., number of people by age range, by gender, or by vehicle type). This consolidation helps create a more holistic, accident-level view, which in turn can facilitate better performance in certain predictive modeling or statistical analysis tasks. (I have done experiment on the dataset with and without grouping, please check the experiment part)
6. **Software and Environment**
    - **Python** (Panda, scikit-learn) for all the data processing.
    - **CSV** format for storing processed data.
    - **Python notebook & excel** for code editing and data inspection.
7. **Hardware**
    - Standard desktop or laptop computer with enough RAM to handle large CSV files. No specialized hardware was required beyond typical development environments.

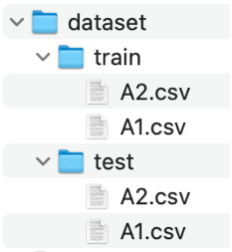
---

4. Data Composition and Amount

4.1 File Structure

There are train and test data inside the dataset folder, and the overall structure looks

like this:



- **A1.csv**: Contains A1 category (fatal accidents) across different years 108(2019) to 113(2022).
- **A2.csv**: Contains A2 category (injury-only) accidents for the same years.

4.2 Rows and Columns

- **Rows**: Each row corresponds to a single accident.
- **Columns**: After processing, the dataset has the following columns:
    1. date, time: When the accident occured (time format HHMMSS)
    2. weather: The mapping of different weather type 晴: 0, 陰: 1, 雨: 2, 霧或煙: 3, 風: 4, 風沙: 5, 雪: 6, 強風: 7, 暴雨: 8
    3. lighting: The mapping of lighting 有照明未開啟或故障: 0, 無照明: 1, 有照明且開啟: 2, 日間自然光線: 3, 夜間(或隧道、地下道、涵洞)有照明: 4, 夜間(或隧道、地下道、涵洞)無照明: 5, 晨或暮光: 6
    4. speed_limit: the speed limit of the road where accident occured.
    5. road_condition: The mapping of different road condition: 乾燥: 0, 濕潤: 1, 泥濘: 2, 油滑: 3, 冰雪: 4
    6. traffic_sign: The mapping of different traffic sign: 無號誌: 0, 閃光號誌: 1, 行車管制號誌(附設行人專用號誌): 2, 行車管制號誌: 3
    7. male, female: The number of males & females involved in the accident
    8. veh_* (e.g. veh_scooter): Number of that vehicle type in the accident.
    9. age_*(e.g. age_21~30): Number of people in the range of that age involved in the accident.

Note: All values are integers (int).

4.3 Data Volume

- A1: 10399 (70% in train folder, 30% in test folder)
- A2: 1583818(70% in train folder, 30% in test folder)

---

5. Data Collection Conditions

1. **Time Frame**: Data covers the years 108 to 113(2019 to 2024)
2. **Geographical Coverage**: All reported accidents occurred within Taiwan.
3. **Accident Severity**: Two categories:
    - A1 (fatal accidents)
    - A2 (injury-only accidents)
4. **Filtering Criteria**:
    - Only rows whose categorical columns match the predefined mappings (e.g., 天候名稱 in [晴, 陰, 雨, 霧或煙, 風, 風沙, 雪, 強風, 暴雨]) are retained.

- Rows with invalid or missing data in the mapped columns are excluded.

---

6. Examples

Here I'll show a row of dataset for example from A1 category(A2 category is in same format):

date	time	weather	lighting	speed_limit	road_condition	traffic_signal	male	female
20240130	61900	0	0	50	0	3	1	1

In this data record, this fatal (A1) accident happened in 2024/01/30 06:19:00. Weather is 晴. Lighting is 有照明未開啟或故障. The speed limit of the road where accident happened is 50 km/h with road condition 乾燥 and traffic signal 行車管制號誌. One male and one female are involved in this accident.

veh_Scooter	veh_Bus	veh_Pedestrian	veh_Truck	veh_Car	veh_Slow	veh_Small_Truck	veh_Full_trailer	veh_Tractor	veh_Semi_trailer	veh_Other	veh_Special	veh_Military
1	0	0	0	1	0	0	0	0	0	0	0	0

In columns starting with veh_, we can see that there are one scooter and one car involved in this accident.

age_1~10	age_11~20	age_21~30	age_31~40	age_41~50	age_51~60	age_61~70	age_71~80	age_81~90	age_91~100	age_101~110	age_111~120
0	0	1	0	0	0	1	0	0	0	0	0

These columns are the distribution of age of those people. From this, we can see that one of that is inside the range 21~30 years old and the other is in the range 61~70.

Final Remarks

This documentation should give you a clear understanding of the dataset's structure, provenance, processing steps, and content. If you need further details—such as additional mappings, year-by-year record counts, or a breakdown of accident severities—please check appendix and experiment sections accordingly.

Algorithm & Analysis

0. Read data & Preprocessing

Before any implementation of any algorithm, we need to do these steps of data reading and preprocessing. First we read the train and test csv file into pandas dataframe. Since there's a large difference in the data amount of A1 and A2 category, we need to do oversampling to Aa category using RandomOverSampler from sklearn. This approach ensures that the model does not develop a bias towards the majority

class (A2), leading to improved generalization. After dealing with data imbalance, I add a feature scaling using StandardScaler from sklearn. A StandardScaler is applied to transform features into a normal distribution with zero mean and unit variance. It ensures that all input features are on a similar scale, preventing any particular feature from dominating others.

---

1. Deep artificial neural network

In the first approach, I perform Stratified (sklearn) K-Fold Cross-Validation on an artificial neural network. Then, use tesorflow keras api to build a fully connected deep artificial neural network.

Model Architecture

- Input Layer: Accepts the standardized input features.
- Hidden Layers: The model employs four hidden layers with ReLU activation functions, batch normalization, and dropout regularization to prevent overfitting.
- Output Layer: A single neuron with a sigmoid activation function is used for binary classification, outputting a probability score between 0 and 1.

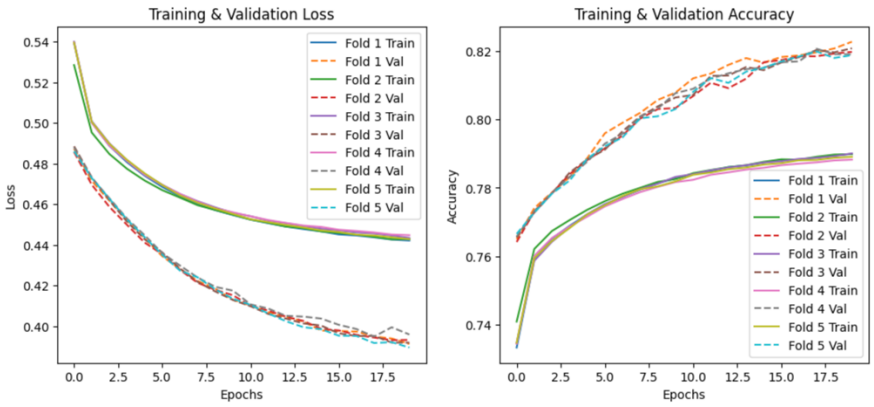
To enhance the model's ability to generalize, batch normalization is applied after each dense layer. Dropout regularization is also implemented to reduce overfitting by randomly deactivating neurons during training.

Model Compilation and Training

The model is compiled using the Adam optimizer, which adapts the learning rate dynamically to optimize convergence. The binary cross-entropy loss function is used to measure the difference between predicted and actual labels.

In each fold, the model is trained for 20 epochs with a batch size of 2048, and validation is conducted using a 30% validation split from the training set. The key performance metrics tracked include accuracy, precision, recall, F1-score, and AUC.

Here’s the loss and accuracy during training of every fold.



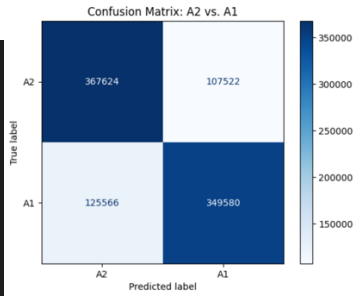
Result(on test data):

Here's the result using classification_report from sklearn. We can see that the result accuracy a little bit higher than 75%. Below is the result of predicting the label with test data:

```
Classification Report:
              precision    recall  f1-score   support

 A2 (Injury)       0.75      0.77      0.76    475146
  A1 (Fatal)       0.76      0.74      0.75    475146

    accuracy                           0.75    950292
   macro avg       0.76      0.75      0.75    950292
weighted avg       0.76      0.75      0.75    950292
```



For better visualization, here's the confusion matrix. We can see that the model did a good job on recognize A2 category. And it can correctly identify more than half of A1 category.

---

2. Decision tree

In the second supervised approach, I choose decision tree for its interpretability and simplicity. Decision trees excel at capturing non-linear relationships in the data, So it should handle this dataset effectively.

Model Architecture

In this approach, we use a 5-fold stratified cross-validation (StratifiedKFold) to maintain the class distribution in each fold. In every fold, we simply use DecisionTreeClassifier from sklearn with max_depth=10. Then we fit the training data to train the classifier model. Finally, we analyze the performance of the model.

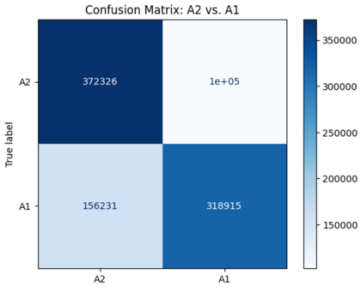
Result(on test data):

Here's the result using classification_report from sklearn. Overall, the model achieves around 77% accuracy. It performs slightly better at identifying "Injury" cases (higher recall) than "Fatal" cases, but it has slightly higher precision for the "Fatal" class.

```
Classification Report:
              precision    recall  f1-score   support

 A2 (Injury)       0.70      0.78      0.74    475146
  A1 (Fatal)       0.76      0.67      0.71    475146

    accuracy                           0.73    950292
   macro avg       0.73      0.73      0.73    950292
weighted avg       0.73      0.73      0.73    950292
```



For better visualization, here's the confusion matrix. We can easily see that decision tree performs better than the previous ann approach. Both A1 and A2 category can get accuracy than 75%.

---

2. Cluster

In this approach, the overall goal was to separate two types of cases—A1 (Fatal) and A2 (Injury)—by applying a clustering method rather than a direct supervised classification.

Model Architecture

Here, I use K-Means from sklearn.cluster. K-Means was set to find two clusters, aligning with the expectation of two natural groups (A1 vs. A2). Once the algorithm partitioned the data into two clusters, each cluster was assigned a label (either A1 or A2) based on the majority of training samples within that cluster. This step transforms the unsupervised cluster labels into meaningful class labels.

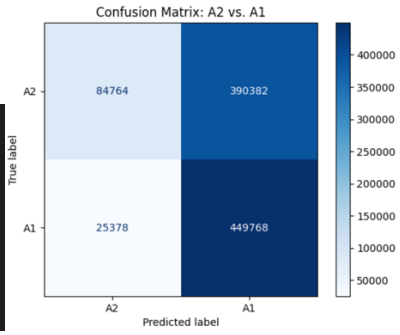
Result(on test data):

Here's the result using classification_report from sklearn. A2 (Injury) exhibits a relatively high precision (0.77) but a very low recall (0.18), which means that when the model predicts A2, it is often correct, but it fails to capture most actual A2 instances. A1 (Fatal) has a lower precision (0.54) but a high recall (0.95). This indicates that the majority of actual A1 cases are correctly identified as such. However, the precision is modest, suggesting that a fair number of A2 cases get mislabeled as A1.

```
Cluster to Label Mapping: {np.int32(0): 1, np.int32(1): 0}
              precision    recall  f1-score   support

 A2 (Injury)       0.77      0.18      0.29    475146
  A1 (Fatal)       0.54      0.95      0.68    475146

    accuracy                           0.56    950292
   macro avg       0.65      0.56      0.49    950292
weighted avg       0.65      0.56      0.49    950292
```



The accuracy of around 0.56 indicates that, overall, the clustering approach is correctly labeling just over half of the cases. It's not as good as previous two supervised learning method.

Experiments

1. Using non processed data (experiment/experiment_ann_original_data.ipynb)

The original data downloaded from open data web by the government is one person per row, which doesn't make sense if we want to predict the severity of the accident. So, here, I did a experiment to compare the performance difference of using the original data and my dataset. Here, we use same deep ann for comparison.

Results (left: original data, right: my dataset):

From the results below, it's obvious that my dataset has led to a more balanced and overall stronger model performance. The original model was highly biased toward

predicting “Injury,” leading to extremely low recall for “Fatal.” The modified dataset (and presumably any associated model changes) produced a better balance: higher recall for “Fatal” and improved precision for “Injury,” thus lifting overall accuracy and F1-scores.

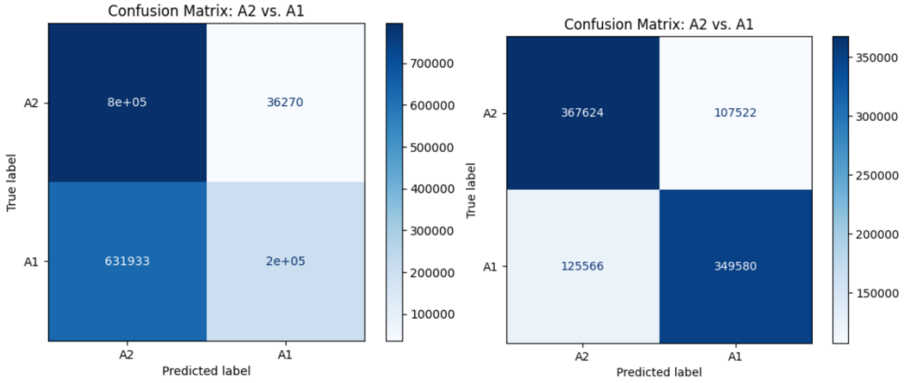
Classification Report:

	precision	recall	f1-score	support
A2 (Injury)	0.56	0.96	0.70	832019
A1 (Fatal)	0.85	0.24	0.37	832019
accuracy			0.60	1664038
macro avg	0.70	0.60	0.54	1664038
weighted avg	0.70	0.60	0.54	1664038

Classification Report:

	precision	recall	f1-score	support
A2 (Injury)	0.75	0.77	0.76	475146
A1 (Fatal)	0.76	0.74	0.75	475146
accuracy			0.75	950292
macro avg	0.76	0.75	0.75	950292
weighted avg	0.76	0.75	0.75	950292

From the confusion matrix of result using original data (left), we can see that in most test data, the model just predict it as A2 label. Overall this result is pretty bad.



2. Amount of training data(experiment/experiment_decision_tree.ipynb)

In the Algorithm and Analysis part, I use all of the training data which is around 1.1 million records. To see if we can increase the speed without losing much accuracy. I do the experiment of different amount of training data. Here we use decision tree as algorithm, then I will try 1%, 10% and 50% of data.

- 1% (around 11000 records): In this amount of data, I found that using smaller max_depth will give a better result. Using max_depth=10(left) we get around 59% accuracy, while max_depth=5 we get 67%. This might because that small data amount might cause overfitting if we choose too large max_depth.

Classification Report:

	precision	recall	f1-score	support
A2 (Injury)	0.56	0.86	0.68	475146
A1 (Fatal)	0.70	0.33	0.45	475146
accuracy			0.59	950292
macro avg	0.63	0.59	0.56	950292
weighted avg	0.63	0.59	0.56	950292

Classification Report:

	precision	recall	f1-score	support
A2 (Injury)	0.78	0.49	0.60	475146
A1 (Fatal)	0.63	0.86	0.73	475146
accuracy			0.67	950292
macro avg	0.70	0.67	0.66	950292
weighted avg	0.70	0.67	0.66	950292

- 10% (around 110000 records): Here we start to get results that are not that bad. But still using smaller max_depth will give a better result. Using max_depth=10(left) we get around 69% accuracy, while max_depth=5 we get 71%. And this is much closer to accuracy using full dataset (73%).

Classification Report:

	precision	recall	f1-score	support
A2 (Injury)	0.69	0.71	0.70	475146
A1 (Fatal)	0.70	0.68	0.69	475146
accuracy			0.69	950292
macro avg	0.69	0.69	0.69	950292
weighted avg	0.69	0.69	0.69	950292

Classification Report:

	precision	recall	f1-score	support
A2 (Injury)	0.78	0.58	0.66	475146
A1 (Fatal)	0.66	0.84	0.74	475146
accuracy			0.71	950292
macro avg	0.72	0.71	0.70	950292
weighted avg	0.72	0.71	0.70	950292

- 50% (around 550000 records): This is very close to the accuracy we achieved using full dataset. And the computing speed is much faster.

Classification Report:

	precision	recall	f1-score	support
A2 (Injury)	0.72	0.72	0.72	475146
A1 (Fatal)	0.72	0.72	0.72	475146
accuracy			0.72	950292
macro avg	0.72	0.72	0.72	950292
weighted avg	0.72	0.72	0.72	950292

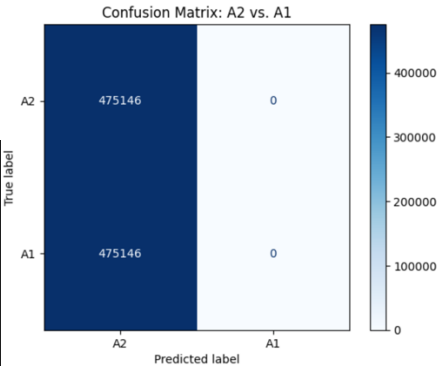
Conclusion: If we can accept a little bit performance reduce, we can get save much computing time. And this means that the dataset doesn't actually need that much of data to get a reasonable result.

3. Oversampling (experiment/experiment_ann_without_oversampling.ipynb)

In this part of experiment, I did an experiment to check the necessity of oversampling. It's obvious that we will need oversampling on the training data due to highly imbalance of amount of the two different categories. Here I use the same ann code but without random oversampling. From the result we can see that the performance is really bad. Basically, the model will predict everything as A2 because A2 has almost 100 times more than A1. So, this means that perform a oversampling is essential for machine learning using this dataset.

Classification Report:

	precision	recall	f1-score	support
A2 (Injury)	0.50	1.00	0.67	475146
A1 (Fatal)	0.00	0.00	0.00	475146
accuracy			0.50	950292
macro avg	0.25	0.50	0.33	950292
weighted avg	0.25	0.50	0.33	950292



Discussion

These experimental results aligned closely with expectations. Aggregating per-person data into meaningful accident-level summaries consistently improved model balance and overall performance compared to using raw data. Models such as deep neural networks and decision trees achieved accuracy higher than 70% after addressing class imbalance through random oversampling. Conversely, clustering methods were

less effective due to their unsupervised nature and sensitivity to the skewed distribution of accident severity (fatal vs. injury-only).

Model performance was notably influenced by dataset imbalance, emphasizing the importance of techniques like oversampling to avoid bias toward the majority class. Effective feature engineering—such as summarizing accident details like vehicle types and age distributions—significantly enhanced predictive accuracy. Additionally, individual feature characteristics (weather, lighting, road conditions) and preprocessing steps like feature scaling were critical to ensuring input data consistency and improving model results.

If more time were available, several additional experiments could further enhance the analysis:
1. Advanced Oversampling Techniques: Investigate methods like SMOTE or ADASYN to generate synthetic samples for the minority class, potentially leading to more robust model training. Actually, I had tried SMOTE. Because I doesn't have much knowledge about it, I get bad results and ended up using random oversampling. So, I might need more understand of that.
2. Hyperparameter Tuning and Ensemble Methods: Explore deeper neural network architectures and tune hyperparameters systematically. Similarly, ensemble methods such as random forests or gradient boosting could be tested to determine if they offer further performance gains.
3. Feature Engineering Enhancements: Experiment with creating interaction terms or higher-order features, and possibly incorporate external data (e.g., real-time traffic or environmental data) to capture additional nuances.
Through these experiments, we learned that careful preprocessing and thoughtful feature aggregation are vital for effectively predicting accident severity. Our results underscore the importance of balancing the dataset and selecting the appropriate model based on the problem's specifics. However, questions remain about how these models might perform under different or evolving conditions, such as in a real-time monitoring environment or with additional external factors. Future work will need to address these questions to further refine the predictive capabilities and operational utility of the models.

Links:
Github link: https://github.com/Luluboy168/Taiwan_traffic_accident_prediction
Raw data link(too large to put on github): https://gofile.me/7srmc/bhtAxWiQC

Appendix

Creating Dataset

data_rename.ipynb:

```
# 統一格式
import os

for year in [108, 109, 110]:
    directory = f"raw_data/{year}"

    for filename in os.listdir(directory):
        if filename.startswith(f"{year + 1911}"):
            new_filename = filename.replace(f"{year + 1911}", str(year), 1)
            old_path = os.path.join(directory, filename)
            new_path = os.path.join(directory, new_filename)
            os.rename(old_path, new_path)
            print(f'Renamed: {filename} -> {new_filename}')
```

data_combine.ipynb

Combine data

```
import pandas as pd
import os
```

Combine monthly data for each year

```
years = [108, 109, 110, 111, 112, 113]

for year in years:
    # Folder path for each year
    folder_path = f"raw_data/{year}"

    # Find all CSV files in that year's folder
    csv_files = [f"{year}年度A2交通事故資料_{i}.csv" for i in range(1, 13)]

    # Read each CSV and store in a list of DataFrames
    df_list = []
    for file in csv_files:
        filename = os.path.join(folder_path, file)
        df_temp = pd.read_csv(filename, encoding="utf-8", low_memory=False)
        df_list.append(df_temp)

    # Combine all monthly DataFrames into one for the year
    df_combined = pd.concat(df_list, ignore_index=True)

    # Save file
    output_filename = f"raw_data/{year}/{year}年度A2交通事故資料.csv"
    df_combined.to_csv(output_filename, index=False, encoding="utf-8-sig")

    print(f"Combined CSV for {year} saved to: {output_filename}")
```

Combine data from different year

```
years = [108, 109, 110, 111, 112, 113]
for i in [1, 2]:
    csv_files = [f"raw_data/{year}/{year}年度A{i}交通事故資料.csv" for year in range(108, 114)]
    df_list = []
    for file in csv_files:
        df_temp = pd.read_csv(file, encoding="utf-8", low_memory=False)
        df_list.append(df_temp)

    df_combined = pd.concat(df_list, ignore_index=True)

    output_filename = f"raw_data/A{i}_raw.csv"
    df_combined.to_csv(output_filename, index=False, encoding="utf-8-sig")

    print(f"Combined CSV for A{i} saved to: {output_filename}")
```

data_processing.ipynb

```python
import pandas as pd
import numpy as np
import os
from utils import mapping
from sklearn.model_selection import train_test_split
```

```python
cols_to_keep = [
    "發生日期",
    "發生時間",
    "事故類別名稱",
    "天候名稱",
    "光線名稱",
    "速限-第1當事者",
    "路面狀況-路面狀態名稱",
    "號誌-號誌種類名稱",
    "當事者區分-類別-大類別名稱-車種",
    "當事者屬-性-別名稱",
    "當事者事故發生時年齡"
]
```

```python
vehicle_types = ["Scooter", "Bus", "Pedestrian", "Truck", "Car", "Slow", "Small_Truck", "Full_trailer", "Tractor", "Semi_trailer", "Other", "Special", "Military"]
vehicle_columns = [f"veh_{vt}" for vt in vehicle_types]
genders = ["男", "女"]

for i in [1, 2]:

    df = pd.read_csv(
        f"raw_data/A{i}_raw.csv",
        encoding="utf-8",
        low_memory=False)

    df = df[cols_to_keep]
    df = df.dropna(how='any')

    df = mapping(df)

    df = df[df["當事者區分-類別-大類別名稱-車種"].isin(vehicle_types)]
    df = df[df["當事者屬-性-別名稱"].isin(genders)]

    # one-hot encoding
    vehicle_dummies = pd.get_dummies(df["當事者區分-類別-大類別名稱-車種"], prefix="veh")
    vehicle_dummies = vehicle_dummies.reindex(columns=vehicle_columns, fill_value=0)

    df["male"] = df["當事者屬-性-別名稱"].apply(lambda x: 1 if x == "男" else 0)
    df["female"] = df["當事者屬-性-別名稱"].apply(lambda x: 1 if x == "女" else 0)

    bins = range(0, 121, 10)
    labels = [f"{i+1}~{i+10}" for i in bins[:-1]]

    df["age_bin"] = pd.cut(
        df["當事者事故發生時年齡"],
        bins=bins,
        labels=labels,
        right=True,
        include_lowest=True
    )

    age_dummies = pd.get_dummies(df["age_bin"], prefix="age")

    df = pd.concat([df, vehicle_dummies, age_dummies], axis=1)
```

```python
# group same accident
agg_dict = {
    "天候名稱": "first",
    "光線名稱": "first",
    "速限-第1當事者": "first",
    "路面狀況-路面狀態名稱": "first",
    "號誌-號誌種類名稱": "first",
    "male": "sum",
    "female": "sum",
}

for col in vehicle_dummies.columns:
    agg_dict[col] = "sum"

for col in age_dummies.columns:
    agg_dict[col] = "sum"


df_grouped = df.groupby(["發生日期", "發生時間"], as_index=False).agg(agg_dict)


df_grouped = df_grouped.rename(columns={
    "發生日期": "date",
    "發生時間": "time",
    "事故類別名稱": "category",
    "天候名稱": "weather",
    "光線名稱": "lighting",
    "速限-第1當事者": "speed_limit",
    "路面狀況-路面狀態名稱": "road_condition",
    "號誌-號誌種類名稱": "traffic_signal",
    "當事者區分-類別-大類別名稱-車種": "vehicle_type",
    "當事者屬-性-別名稱": "gender",
    "當事者事故發生時年齡": "age",
})

df_grouped["date"] = df_grouped["date"].astype(int)
df_grouped["time"] = df_grouped["time"].astype(int)
df_grouped["speed_limit"] = df_grouped["speed_limit"].astype(int)

df_grouped.to_csv(f"raw_data/A{i}.csv", index=False, encoding="utf-8-sig")
```

```python
os.makedirs("dataset/train", exist_ok=True)
os.makedirs("dataset/test", exist_ok=True)

for i in [1, 2]:
    df = pd.read_csv(f"raw_data/A{i}.csv")
    print(df.shape)

    train_df, test_df = train_test_split(df, test_size=0.3, random_state=42)

    train_df.to_csv(f"dataset/train/A{i}.csv", index=False)
    test_df.to_csv(f"dataset/test/A{i}.csv", index=False)

    print(f"Dataset split into train and test sets and saved to 'dataset/train/A{i}.csv' and 'dataset/test/A{i}.csv'.")
```

Algorithms

ann.ipynb

```python
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import precision_score, recall_score, f1_score, classification_report
from sklearn.utils import class_weight
from sklearn.preprocessing import StandardScaler

import tensorflow as tf
from tensorflow.keras import layers
from tensorflow.keras import backend as K

import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

from imblearn.over_sampling import RandomOverSampler
from collections import Counter
```

```python
def load_data(type, file_type):
    path = f"dataset/{type}/{file_type}.csv"
    df = pd.read_csv(
        path,
        encoding="utf-8",
        low_memory=False,
        dtype=int)

    df["label"] = 1 if file_type == "A1" else 0
    return df
```

```python
train_dfs_A1 = load_data("train", "A1")
train_dfs_A2 = load_data("train", "A2")

df_train = pd.concat([train_dfs_A1, train_dfs_A2], ignore_index=True)

df_a1_test = load_data("test", "A1")
df_a2_test = load_data("test", "A2")

# Combine into one DataFrame for testing
df_test = pd.concat([df_a1_test, df_a2_test], ignore_index=True)

# shuffle
df_train = df_train.sample(frac=1, random_state=42)
df_test = df_test.sample(frac=1, random_state=42)

cols = [col for col in df_train.columns if (col != '發生日期' and col != "label")]

df_train = df_train.dropna(how='any')
df_test = df_test.dropna(how='any')

X_train = df_train[cols]
y_train = df_train["label"]

X_test = df_test[cols]
y_test = df_test["label"]
```

(property) columns: Index[str]

```python
# Print original class distribution
print("Original distribution:", Counter(y_train))

# Oversample the training data
ros = RandomOverSampler(sampling_strategy='minority')
X_train, y_train = ros.fit_resample(X_train, y_train)
X_test, y_test = ros.fit_resample(X_test, y_test)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Check the new class distribution
print("Resampled distribution:", Counter(y_train))
```

```python
class F1Score(tf.keras.metrics.Metric):
    def __init__(self, name='f1_score', threshold=0.5, **kwargs):
        super(F1Score, self).__init__(name=name, **kwargs)
        self.threshold = threshold
        self.true_positives = self.add_weight(name='tp', initializer='zeros')
        self.false_positives = self.add_weight(name='fp', initializer='zeros')
        self.false_negatives = self.add_weight(name='fn', initializer='zeros')

    def update_state(self, y_true, y_pred, sample_weight=None):
        # Convert probabilities to binary predictions
        y_pred = tf.cast(tf.greater(y_pred, self.threshold), tf.float32)
        y_true = tf.cast(y_true, tf.float32)

        tp = tf.reduce_sum(y_true * y_pred)
        fp = tf.reduce_sum((1 - y_true) * y_pred)
        fn = tf.reduce_sum(y_true * (1 - y_pred))

        self.true_positives.assign_add(tp)
        self.false_positives.assign_add(fp)
        self.false_negatives.assign_add(fn)

    def result(self):
        precision = self.true_positives / (self.true_positives + self.false_positives + K.epsilon())
        recall = self.true_positives / (self.true_positives + self.false_negatives + K.epsilon())
        return 2 * ((precision * recall) / (precision + recall + K.epsilon()))

    def reset_states(self):
        self.true_positives.assign(0)
        self.false_positives.assign(0)
        self.false_negatives.assign(0)
```

```python
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
import numpy as np

num_folds = 5
skf = StratifiedKFold(n_splits=num_folds, shuffle=True, random_state=42)
fold_no = 1

# Initialize lists to store classification reports, confusion matrices, and histories
all_reports = []
conf_matrices = []
histories = []

for train_index, val_index in skf.split(X_train, y_train):
    print(f"Training on fold {fold_no}/{num_folds}...")

    # Split data
    X_train_fold, X_val_fold = X_train[train_index], X_train[val_index]
    y_train_fold, y_val_fold = y_train[train_index], y_train[val_index]

    # Create a new ANN model for each fold
    model = tf.keras.Sequential([
        layers.Dense(64, activation="relu", input_shape=(X_train.shape[1],)),
        layers.BatchNormalization(),
        layers.Dropout(0.3),

        layers.Dense(128, activation="relu"),
        layers.BatchNormalization(),
        layers.Dropout(0.3),

        layers.Dense(64, activation="relu"),
        layers.BatchNormalization(),
        layers.Dropout(0.3),

        layers.Dense(32, activation="relu"),
        layers.BatchNormalization(),
        layers.Dropout(0.3),

        layers.Dense(16, activation="relu"),
        layers.BatchNormalization(),
        layers.Dropout(0.2),

        layers.Dense(1, activation="sigmoid")
    ])
```

```python
    model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=[
        tf.keras.metrics.BinaryAccuracy(name="accuracy"),
        tf.keras.metrics.Precision(name='precision'),
        tf.keras.metrics.Recall(name='recall'),
        F1Score(name='f1_score'),
        tf.keras.metrics.AUC(name='auc', num_thresholds=200)
    ]
)

    # Train the model and store history
    history = model.fit(X_train_fold,
                        y_train_fold,
                        epochs=20,
                        batch_size=2048,
                        validation_data=(X_val_fold, y_val_fold),
                        verbose=1)
    histories.append(history)

    # Predict validation set
    y_pred_probs = model.predict(X_val_fold)
    y_pred = (y_pred_probs > 0.5).astype(int)

    # Store classification report
    report = classification_report(y_val_fold, y_pred, output_dict=True)
    all_reports.append(report)

    # Store confusion matrix
    conf_matrices.append(confusion_matrix(y_val_fold, y_pred))

    fold_no += 1
```

```python
# Plot training history for loss and accuracy
plt.figure(figsize=(12, 5))

# Loss plot
plt.subplot(1, 2, 1)
for i, history in enumerate(histories):
    plt.plot(history.history["loss"], label=f"Fold {i+1} Train")
    plt.plot(history.history["val_loss"], linestyle="dashed", label=f"Fold {i+1} Val")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Training & Validation Loss")
plt.legend()

# Accuracy plot
plt.subplot(1, 2, 2)
for i, history in enumerate(histories):
    plt.plot(history.history["accuracy"], label=f"Fold {i+1} Train")
    plt.plot(history.history["val_accuracy"], linestyle="dashed", label=f"Fold {i+1} Val")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.title("Training & Validation Accuracy")
plt.legend()

plt.show()
```

```python
pred_probs = model.predict(X_test).ravel()
pred_labels = (pred_probs > 0.5).astype(int)

precision = precision_score(y_test, pred_labels)
recall = recall_score(y_test, pred_labels)
f1 = f1_score(y_test, pred_labels)

print("Precision:", precision)
print("Recall:   ", recall)
print("F1 Score: ", f1)

# Optional: detailed classification report
print("\nClassification Report:")
print(classification_report(y_test, pred_labels, target_names=["A2 (Injury)", "A1 (Fatal)"]))
```

```python
# 1. Compute the confusion matrix
cm = confusion_matrix(y_test, pred_labels)

# 2. Create a display object,  passing in class names label=0 -> A2, label=1 -> A1
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=["A2", "A1"])

# 3. Plot the confusion matrix
disp.plot(cmap=plt.cm.Blues)
plt.title("Confusion Matrix: A2 vs. A1")
plt.show()
```

decision_tree.ipynb

```python
import pandas as pd
import numpy as np

from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score
from sklearn.preprocessing import StandardScaler

import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

from imblearn.over_sampling import RandomOverSampler
from collections import Counter


from sklearn.model_selection import StratifiedKFold
```

```python
def load_data(type, file_type):
    path = f"dataset/{type}/{file_type}.csv"
    df = pd.read_csv(
        path,
        encoding="utf-8",
        low_memory=False,
        dtype=int)

    df["label"] = 1 if file_type == "A1" else 0
    return df
```

```python
train_dfs_A1 = load_data("train", "A1")
train_dfs_A2 = load_data("train", "A2")

df_train = pd.concat([train_dfs_A1, train_dfs_A2], ignore_index=True)

df_a1_test = load_data("test", "A1")
df_a2_test = load_data("test", "A2")

# Combine into one DataFrame for testing
df_test = pd.concat([df_a1_test, df_a2_test], ignore_index=True)

# shuffle
df_train = df_train.sample(frac=1, random_state=42)
df_test = df_test.sample(frac=1, random_state=42)

cols = [col for col in df_train.columns if (col != '發生日期' and col != "label")]

df_train = df_train.dropna(how='any')
df_test = df_test.dropna(how='any')

X_train = df_train[cols]
y_train = df_train["label"]

X_test = df_test[cols]
y_test = df_test["label"]
```

```python
# Print original class distribution
print("Original distribution:", Counter(y_train))

# Oversample the training data
ros = RandomOverSampler(sampling_strategy='minority')
X_train, y_train = ros.fit_resample(X_train, y_train)
X_test, y_test = ros.fit_resample(X_test, y_test)

# Check the new class distribution
print("Resampled distribution:", Counter(y_train))

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```python
num_folds = 5
skf = StratifiedKFold(n_splits=num_folds, shuffle=True, random_state=42)
fold_no = 1

# Initialize lists to store classification reports
all_reports = []
conf_matrices = []

for train_index, val_index in skf.split(X_train, y_train):
    print(f"Training on fold {fold_no}/{num_folds}...")

    # Split data
    X_train_fold, X_val_fold = X_train[train_index], X_train[val_index]
    y_train_fold, y_val_fold = y_train[train_index], y_train[val_index]

    # Create and train the Decision Tree model
    clf = DecisionTreeClassifier(random_state=42, max_depth=10)
    clf.fit(X_train_fold, y_train_fold)

    # Predict validation set
    y_pred = clf.predict(X_val_fold)

    # Store classification report
    report = classification_report(y_val_fold, y_pred, output_dict=True)
    all_reports.append(report)

    # Store confusion matrix
    conf_matrices.append(confusion_matrix(y_val_fold, y_pred))

    fold_no += 1

# Compute average classification report
avg_report = {}
for key in all_reports[0].keys():
    if key not in ["accuracy", "macro avg", "weighted avg"]:
        avg_report[key] = {
            "precision": np.mean([rep[key]["precision"] for rep in all_reports]),
            "recall": np.mean([rep[key]["recall"] for rep in all_reports]),
            "f1-score": np.mean([rep[key]["f1-score"] for rep in all_reports]),
            "support": int(np.mean([rep[key]["support"] for rep in all_reports])),
        }

# Add macro avg and weighted avg
avg_report["macro avg"] = {
    "precision": np.mean([rep["macro avg"]["precision"] for rep in all_reports]),
    "recall": np.mean([rep["macro avg"]["recall"] for rep in all_reports]),
    "f1-score": np.mean([rep["macro avg"]["f1-score"] for rep in all_reports]),
    "support": int(np.mean([rep["macro avg"]["support"] for rep in all_reports])),
}
```

```python
avg_report["weighted avg"] = {
    "precision": np.mean([rep["weighted avg"]["precision"] for rep in all_reports]),
    "recall": np.mean([rep["weighted avg"]["recall"] for rep in all_reports]),
    "f1-score": np.mean([rep["weighted avg"]["f1-score"] for rep in all_reports]),
    "support": int(np.mean([rep["weighted avg"]["support"] for rep in all_reports])),
}
```

```python
# Make predictions on the test set
predictions = clf.predict(X_test)

# Evaluate the model
print("Accuracy:", accuracy_score(y_test, predictions))
print("\nClassification Report:")
print(classification_report(y_test, predictions, target_names=["A2 (Injury)", "A1 (Fatal)"]))
```

```python
# 1. Compute the confusion matrix
cm = confusion_matrix(y_test, predictions)

# 2. Create a display object, optionally passing in class names
#    Here we assume label=0 -> A2, label=1 -> A1
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=["A2", "A1"])

# 3. Plot the confusion matrix
disp.plot(cmap=plt.cm.Blues)
plt.title("Confusion Matrix: A2 vs. A1")
plt.show()
```

clustering.ipynb

```
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from imblearn.over_sampling import RandomOverSampler
from collections import Counter
```

```
def load_data(type, file_type):
    path = f"dataset/{type}/{file_type}.csv"
    df = pd.read_csv(
        path,
        encoding="utf-8",
        low_memory=False,
        dtype=int)

    df["label"] = 1 if file_type == "A1" else 0
    return df
```

```
train_dfs_A1 = load_data("train", "A1")
train_dfs_A2 = load_data("train", "A2")

df_train = pd.concat([train_dfs_A1, train_dfs_A2], ignore_index=True)

df_a1_test = load_data("test", "A1")
df_a2_test = load_data("test", "A2")

# Combine into one DataFrame for testing
df_test = pd.concat([df_a1_test, df_a2_test], ignore_index=True)

# shuffle
df_train = df_train.sample(frac=1, random_state=42)
df_test = df_test.sample(frac=1, random_state=42)

cols = [col for col in df_train.columns if (col != '發生日期' and col != "label")]

df_train = df_train.dropna(how='any')
df_test = df_test.dropna(how='any')

X_train = df_train[cols]
y_train = df_train["label"]

X_test = df_test[cols]
y_test = df_test["label"]
```

```
# Print original class distribution
print("Original distribution:", Counter(y_train))

# Oversample the training data
ros = RandomOverSampler(sampling_strategy='minority')
X_train, y_train = ros.fit_resample(X_train, y_train)
X_test, y_test = ros.fit_resample(X_test, y_test)

scaler = StandardScaler()
X_train_normalized = scaler.fit_transform(X_train)
X_test_normalized = scaler.transform(X_test)

# Check the new class distribution
print("Resampled distribution:", Counter(y_train))
```

```python
n_clusters = 2  # one for A1 and one for A2
kmeans = KMeans(n_clusters=n_clusters, random_state=42)
clusters_train = kmeans.fit_predict(X_train_normalized)

# ---------------------------------
# Map each cluster to a label based on majority vote
# ---------------------------------
cluster_label_mapping = {}
for cluster in np.unique(clusters_train):
    # Get indices of training samples in the cluster
    indices = np.where(clusters_train == cluster)[0]
    # Count labels in this cluster
    label_counts = Counter(y_train[indices])
    # Determine the majority label in this cluster
    majority_label = label_counts.most_common(1)[0][0]
    cluster_label_mapping[cluster] = majority_label

print("Cluster to Label Mapping:", cluster_label_mapping)

# ---------------------------------
# Use the clustering model to predict clusters for X_test
# ---------------------------------
clusters_test = kmeans.predict(X_test_normalized)

# Map cluster labels to the final predicted class
y_pred = np.array([cluster_label_mapping[cluster] for cluster in clusters_test])

# ---------------------------------
# Evaluate performance
# ---------------------------------
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred, target_names=["A2 (Injury)", "A1 (Fatal)"]))
```

```python
# 1. Compute the confusion matrix
cm = confusion_matrix(y_test, y_pred)

# 2. Create a display object, optionally passing in class names
#    label=0 -> A2, label=1 -> A1
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=["A2", "A1"])

# 3. Plot the confusion matrix
disp.plot(cmap=plt.cm.Blues)
plt.title("Confusion Matrix: A2 vs. A1")
plt.show()
```